

PLAYWRIGHT

Complete Commands Reference

End-to-End Testing Framework

[CLI](#) • [Browser](#) • [Locators](#) • [Assertions](#) • [Network](#) • [More](#)

1. Installation & CLI Commands

Install Playwright and manage browsers from the terminal.

```
# Install Playwright into your project  
npm init playwright@latest
```

```
# Install all browsers  
npx playwright install
```

```
# Install specific browser  
npx playwright install chromium  
npx playwright install firefox  
npx playwright install webkit
```

```
# Run all tests  
npx playwright test
```

```
# Run a specific test file  
npx playwright test tests/example.spec.ts
```

```
# Run tests matching a title  
npx playwright test -g "login page"
```

```
# Headed mode (visible browser window)  
npx playwright test --headed
```

```
# Run in a specific browser  
npx playwright test --project=chromium  
npx playwright test --project=firefox  
npx playwright test --project=webkit
```

```
# Debug mode (step through tests)  
npx playwright test --debug
```

```
# UI Mode (visual test runner)  
npx playwright test --ui
```

```
# Generate HTML report  
npx playwright show-report
```

```
# Record new tests (Codegen)  
npx playwright codegen https://example.com
```

```
# View recorded trace  
npx playwright show-trace trace.zip
```

```
# Run tests in parallel  
npx playwright test --workers=4
```

```
# Retry on failure  
npx playwright test --retries=3
```

```
# Set timeout  
npx playwright test --timeout=60000
```

2. Browser Launch & Context

```
import { chromium, firefox, webkit } from '@playwright/test';

// Launch a browser
const browser = await chromium.launch();
const browser = await chromium.launch({ headless: false }); //
visible
const browser = await chromium.launch({ slowMo: 500 });    //
slow motion
const browser = await chromium.launch({ devtools: true }); //
open DevTools

// Create a browser context
const context = await browser.newContext();
const context = await browser.newContext({
  viewport: { width: 1280, height: 720 },
  userAgent: 'Custom Agent',
  locale: 'en-US',
  timezoneId: 'America/New_York',
  geolocation: { latitude: 40.7128, longitude: -74.0060 },
  permissions: ['geolocation'],
  httpCredentials: { username: 'user', password: 'pass' },
  ignoreHTTPSErrors: true,
  recordVideo: { dir: 'videos/' },
  storageState: 'auth.json',
});

// Open a new page
const page = await context.newPage();

// Close everything
await page.close();
await context.close();
await browser.close();
```

3. Navigation

```
// Go to a URL
await page.goto('https://example.com');
await page.goto('https://example.com', { waitUntil: 'networkidle' });
// waitUntil: 'load' | 'domcontentloaded' | 'networkidle' | 'commit'

// Navigate back / forward
await page.goBack();
await page.goForward();

// Reload the page
await page.reload();

// Wait for URL
await page.waitForURL('https://example.com/dashboard');
await page.waitForURL(/dashboard/);

// Get current URL
const url = page.url();

// Get page title
const title = await page.title();
```

4. Locators

Locators are the preferred way to find elements. They are auto-retrying and auto-waiting.

By Role (Recommended)

```
page.getByRole('button', { name: 'Submit' })
page.getByRole('textbox', { name: 'Email' })
page.getByRole('link', { name: 'Home' })
page.getByRole('checkbox')
page.getByRole('heading', { level: 1 })
page.getByRole('combobox') // <select>
page.getByRole('listitem')
```

By Text, Label & Placeholder

```
page.getByText('Hello World')
page.getByText('Hello', { exact: true })
page.getByLabel('Email address')
page.getByPlaceholder('Enter email')
page.getByAltText('Company logo') // images
page.getByTitle('Tooltip text')
page.getByTestId('submit-btn') // data-testid attribute
```

CSS / XPath / nth

```
page.locator('.my-class')
page.locator('#my-id')
page.locator('input[type="email"]')
page.locator('//button[text()='Submit']') // XPath

page.locator('li').nth(0) // first element
page.locator('li').first()
page.locator('li').last()
```

Filter & Chain

```
page.getByRole('listitem').filter({ hasText: 'Product 1' })
page.locator('.card').filter({ has: page.getByRole('button') })
page.locator('.form').getByRole('button', { name: 'Submit' })
```

5. Interactions

Click, Double-click & Right-click

```
await page.getByRole('button').click();
await page.getByRole('button').click({ button: 'right' });
await page.getByRole('button').click({ clickCount: 2 });
await page.getByRole('button').click({ modifiers: ['Shift'] });
await page.getByRole('button').dblclick();
```

Hover, Focus & Tap

```
await page.getByRole('button').hover();
await page.getByRole('textbox').focus();
await page.getByRole('button').tap(); // mobile
```

Fill, Type & Clear

```
await page.getByLabel('Name').fill('John Doe'); // clears first
await page.getByLabel('Name').pressSequentially('John', { delay:
100 });
await page.getByLabel('Name').clear();
```

Keyboard

```
await page.getByLabel('Search').press('Enter');
await page.getByLabel('Search').press('Control+A');
await page.keyboard.press('Tab');
await page.keyboard.press('Escape');
await page.keyboard.type('Hello World');
await page.keyboard.down('Shift');
await page.keyboard.up('Shift');
```

Select, Checkbox & File

```
await page.getByRole('combobox').selectOption('value');
await page.getByRole('combobox').selectOption({ label: 'Label' });
await page.getByRole('combobox').selectOption(['val1', 'val2']);

await page.getByRole('checkbox').check();
await page.getByRole('checkbox').unchecked();
await page.getByRole('checkbox').setChecked(true);

await page.getByLabel('Upload').setInputFiles('file.pdf');
```

```
await page.getByLabel('Upload').setInputFiles(['f1.pdf', 'f2.pdf']);  
await page.getByLabel('Upload').setInputFiles([]); // clear
```

Drag, Scroll & Mouse

```
await page.dragAndDrop('#source', '#target');  
await page.getByText('Drag me').dragTo(page.getByText('Drop  
here'));
```

```
await page.mouse.wheel(0, 500); // scroll down  
await page.mouse.move(100, 200);  
await page.mouse.down();  
await page.mouse.up();  
await page.mouse.click(100, 200);  
await page.mouse.dblclick(100, 200);
```


6. Assertions (expect)

```
import { expect } from '@playwright/test';

// Visibility
await expect(locator).toBeVisible();
await expect(locator).toBeHidden();

// Text
await expect(locator).toHaveText('Hello');
await expect(locator).toHaveText(/Hello/);
await expect(locator).toContainText('partial text');

// Input value & attribute
await expect(locator).toHaveValue('input value');
await expect(locator).toHaveAttribute('href', 'https://example.com');
await expect(locator).toHaveAttribute('class', /active/);

// Class & CSS
await expect(locator).toHaveClass('active');
await expect(locator).toHaveCSS('color', 'rgb(0,0,0)');

// Count
await expect(locator).toHaveCount(5);

// State
await expect(locator).toBeChecked();
await expect(locator).toBeEnabled();
await expect(locator).toBeDisabled();
await expect(locator).toBeFocused();
await expect(locator).toBeEditable();
await expect(locator).toBeEmpty();

// Page-level
await expect(page).toHaveURL('https://example.com');
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle('My App');

// Soft assertions (don't stop test on failure)
await expect.soft(locator).toBeVisible();
```

```
// Negate
await expect(locator).not.toBeVisible();
await expect(locator).not.toHaveText('Error');

// Screenshot comparison
await expect(page).toHaveScreenshot('baseline.png');
await expect(locator).toHaveScreenshot();
```

7. Waiting

💡 *Playwright auto-waits for elements before interacting. Explicit waits are rarely needed.*

```
// Wait for element state
await page.waitForSelector('.element');
await page.waitForSelector('.element', { state: 'visible' });
// states: 'attached' | 'detached' | 'visible' | 'hidden'

// Wait for navigation
await page.waitForNavigation();
await page.waitForURL('/dashboard');

// Wait for load state
await page.waitForLoadState('networkidle');
await page.waitForLoadState('domcontentloaded');

// Wait for network
await page.waitForRequest('https://api.example.com/users');
await page.waitForResponse('https://api.example.com/users');
await page.waitForResponse(r => r.url().includes('/api') &&
r.status() === 200);

// Wait for JS condition
await page.waitForFunction(() => document.title === 'Loaded');

// Wait for event
await page.waitForEvent('popup');
await page.waitForEvent('download');
await page.waitForEvent('dialog');

// Fixed timeout (avoid when possible)
await page.waitForTimeout(1000);
```

8. Frames & iFrames

```
// Locate elements inside an iframe
await page.frameLocator('#my-iframe').getByRole('button').click();
await
page.frameLocator('[name="checkout"]').getByLabel('Email').fill('a
@b.com');

// Get frame by name
const frame = page.frame('frame-name');

// List all frames
const frames = page.frames();
```

9. Dialogs (alert, confirm, prompt)

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept();      // click OK
  await dialog.accept('text'); // for prompt input
  await dialog.dismiss();     // click Cancel
});
```

10. Network Interception

Abort, Mock & Modify

```
// Block image requests
await page.route('**/*.png', route => route.abort());

// Mock an API response
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ users: [] }),
  });
});

// Modify a request (add header)
await page.route('**/api/data', async route => {
  const req = route.request();
  await route.continue({
    headers: { ...req.headers(), 'X-Custom': 'value' }
  });
});

// Remove route
await page.unroute('**/api/users');
```

Wait for Request / Response

```
const [response] = await Promise.all([
  page.waitForResponse('**/api/users'),
  page.getByRole('button', { name: 'Load' }).click(),
]);
const json = await response.json();
```

11. JavaScript Execution

```
// Run JS in browser context
const title = await page.evaluate(() => document.title);

// Pass arguments
const sum = await page.evaluate(([a, b]) => a + b, [2, 3]);

// Evaluate on a locator
const text = await page.locator('h1').evaluate(el => el.innerText);
const all = await page.locator('li').evaluateAll(els => els.map(e => e.textContent));

// Expose Node.js function to browser
await page.exposeFunction('myFunc', () => 'result from node');

// Run script before page loads
await page.addInitScript(() => { window.myGlobal = 'value'; });
await page.addInitScript({ path: 'preload.js' });
```

12. Screenshots & Videos

```
// Full page screenshot
await page.screenshot({ path: 'screenshot.png', fullPage: true });

// Element screenshot
await page.locator('.card').screenshot({ path: 'card.png' });

// Buffer
const buffer = await page.screenshot();

// Record video (set during context creation)
const context = await browser.newContext({
  recordVideo: { dir: 'videos/' }
});
```

```
// Save video  
await page.close();  
await page.video().saveAs('my-video.webm');
```

13. Tracing

```
// Start tracing
await context.tracing.start({ screenshots: true, snapshots: true });

// Stop and save trace
await context.tracing.stop({ path: 'trace.zip' });

// View trace in browser
// npx playwright show-trace trace.zip
```

14. Storage & Cookies

```
// Save auth state
await context.storageState({ path: 'auth.json' });

// Load auth state when creating context
const context = await browser.newContext({ storageState:
'auth.json' });

// Cookies
await context.addCookies([{ name: 'token', value: 'abc', url:
'https://example.com' }]);
const cookies = await context.cookies();
await context.clearCookies();

// Local storage (via evaluate)
await page.evaluate(() => localStorage.setItem('key', 'value'));
const val = await page.evaluate(() => localStorage.getItem('key'));
```


15. Multi-tab & Popups

```
// Open new tab
const newPage = await context.newPage();

// Handle popup
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link').click(),
]);
await popup.waitForLoadState();
console.log(await popup.title());
```

16. Mobile & Emulation

```
import { devices } from '@playwright/test';

// Use a device preset
const context = await browser.newContext({
  ...devices['iPhone 14'],
});

// Dark mode
const context = await browser.newContext({
  colorScheme: 'dark', // 'light' | 'dark' | 'no-preference'
});

// Offline mode
await context.setOffline(true);

// Geolocation
await context.setGeolocation({ latitude: 51.5, longitude: -0.1 });
await context.grantPermissions(['geolocation']);

// Extra HTTP headers
await page.setExtraHTTPHeaders({ 'Authorization': 'Bearer token'
});
```

17. Test Hooks & Fixtures

```
import { test, expect } from '@playwright/test';

// Lifecycle hooks
test.beforeAll(async () => { /* runs once before all tests */ });
test.afterAll(async () => {});
test.beforeEach(async ({ page }) => { await page.goto('/'); });
test.afterEach(async ({ page }) => {});

// Skip / Only / Fixme
test.skip('skipped test', async () => {});
test.only('run only this', async () => {});
test.fixme('broken test', async () => {});

// Tag tests
test('my test', { tag: '@smoke' }, async ({ page }) => {});

// Group tests
test.describe('Auth Tests', () => {
  test('login', async ({ page }) => {});
  test('logout', async ({ page }) => {});
});

// Custom fixture
const myTest = test.extend({
  myPage: async ({ page }, use) => {
    await page.goto('/login');
    await use(page);
  },
});
```

18. playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  retries: 2,
  workers: 4,
  reporter: 'html',
  use: {
    baseURL: 'https://example.com',
    headless: true,
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile', use: { ...devices['iPhone 14'] } },
  ],
});
```

✓ *Playwright auto-waits for elements — you rarely need manual waits when using modern locators.*